UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Innovación Basada en la Información y el Conocimiento

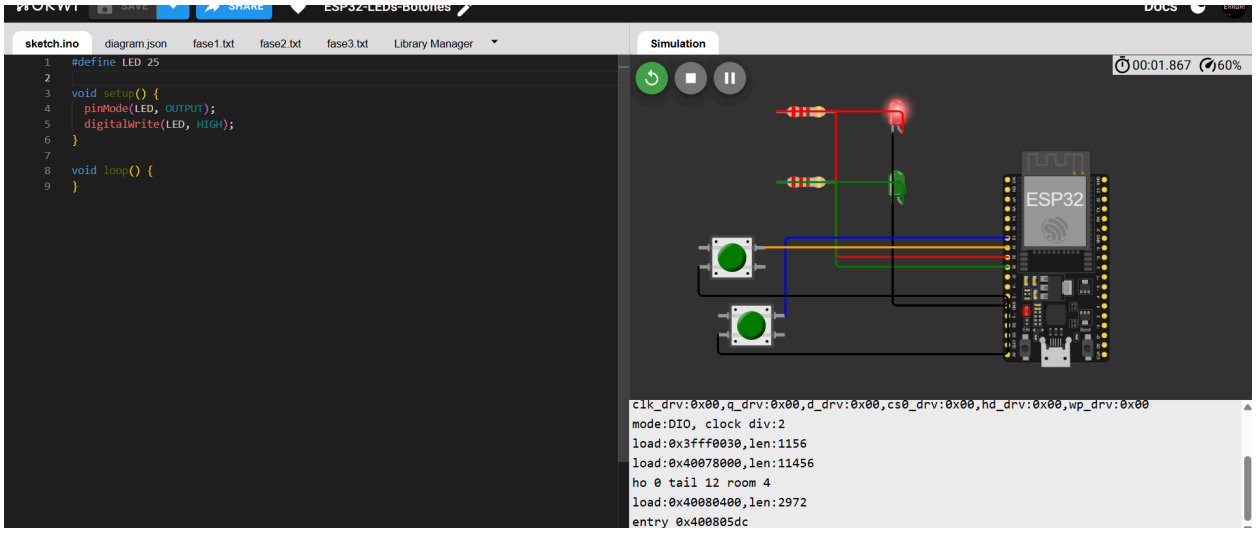


Ingeniería Informática

Nombre: Aguilar Nova Eric Oswaldo		**Código**: 219635295
Nombre de la clase: IoT	**Nombre del profesor**: Mtro. Zeus Cobián	
Atributos de la competencia: ***Experimentación y curiosidad tecnológica***		**Práctica**: 1
Fecha: 29/08/26		



Fase 1



Codigo fase 1

```
/**
 * @file Fase1_LED_Fijo.cpp
 * @brief Práctica 1 - Fase 1: Control de salida digital fija en ESP32.
 * @author Aguilar Nova Eric Oswaldo
 * @date 2026B
 */

#include <Arduino.h>

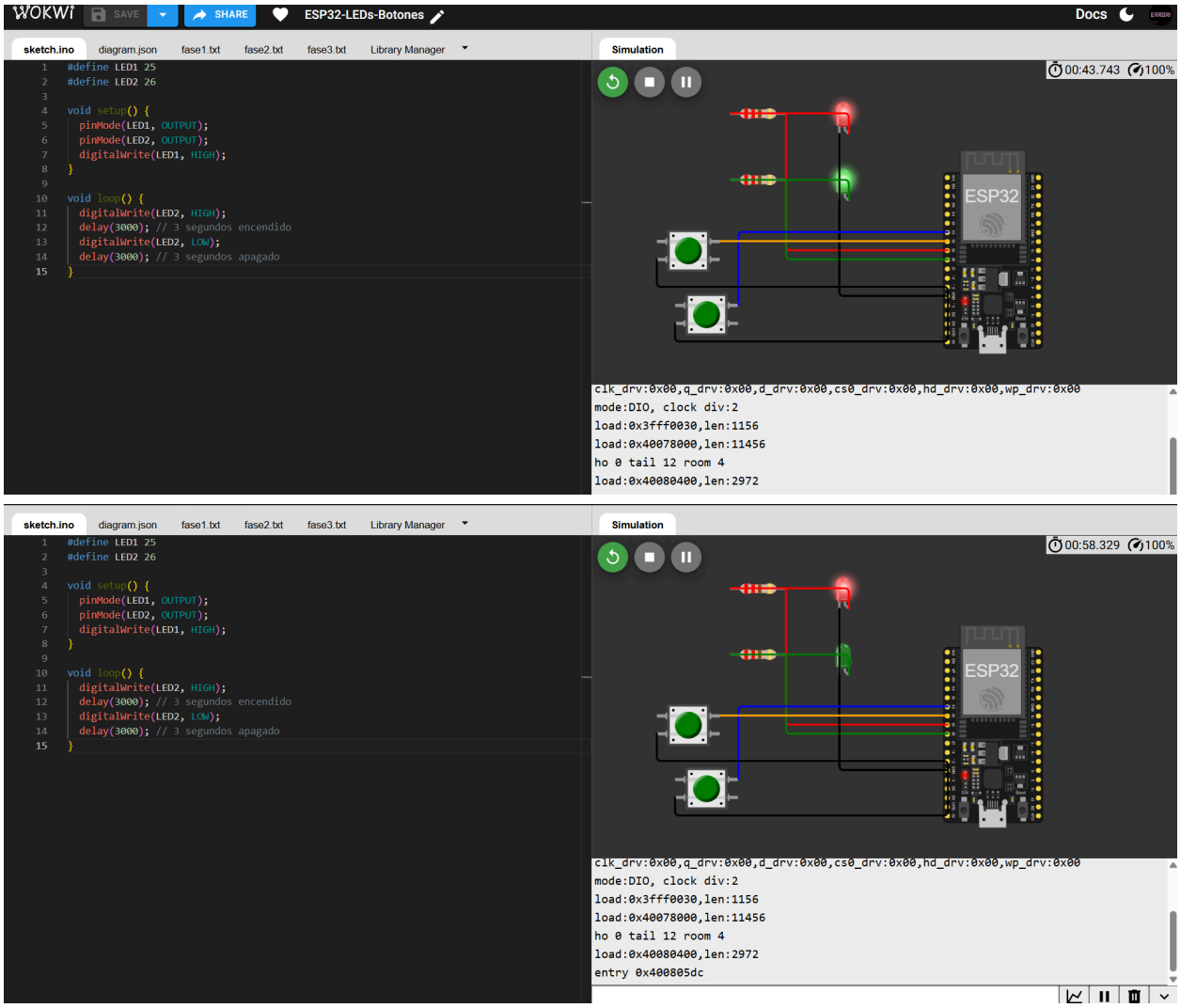
namespace Config {
    constexpr uint8_t PIN_LED_PRINCIPAL {25}; // Pin asignado al LED1
}

/**
 * @brief Inicializa los periféricos del microcontrolador.
 */
void setup() {
    pinMode(Config::PIN_LED_PRINCIPAL, OUTPUT);
    digitalWrite(Config::PIN_LED_PRINCIPAL, HIGH); // Estado permanente encendido
}

/**
```

```
 * @brief Bucle principal de ejecución.
 */
void loop() {
    // Sin procesos de fondo necesarios en la Fase 1
}
```

Fase 2



Codigo fase 2

```
/**
 * @file Fase2_Dual_LED_Asincrono.cpp
 * @brief Práctica 1 - Fase 2: Control concurrente de LEDs mediante tiempo no bloqueante.
 * @author Aguilar Nova Eric Oswaldo
 * @date 2026B
 */
```

```
#include <Arduino.h>


namespace Config {
    constexpr uint8_t PIN_LED_FIJO {25};
    constexpr uint8_t PIN_LED_PARPADEO {26};
    constexpr uint32_t INTERVALO_PARPADEO_MS {3000}; // 3 segundos de
conmutación
}


// Variables de estado no bloqueante
uint32_t tiempoAnterior {0};
bool estadoLed2 {LOW};


/**
 * @brief Gestiona el parpadeo del LED secundario utilizando
millis().
 */
void actualizarLedParpadeante() {
    uint32_t tiempoActual = millis();

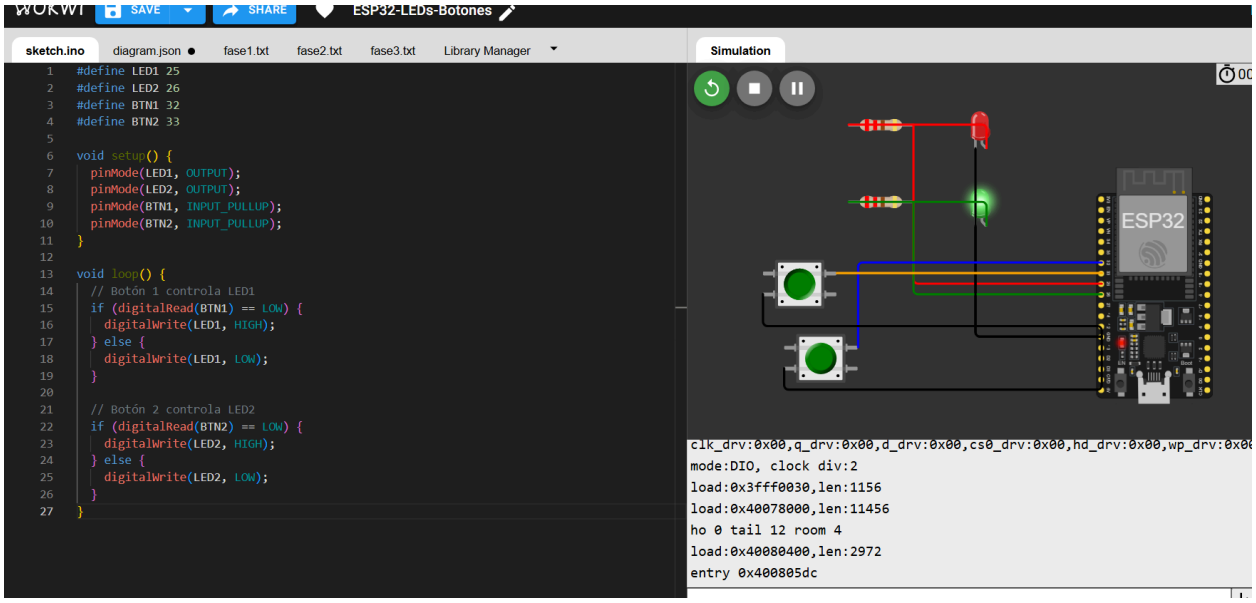

    if (tiempoActual - tiempoAnterior >=
Config::INTERVALO_PARPADEO_MS) {
        tiempoAnterior = tiempoActual;
        estadoLed2 = !estadoLed2;
        digitalWrite(Config::PIN_LED_PARPADEO, estadoLed2 ? HIGH :
LOW);
    }
}

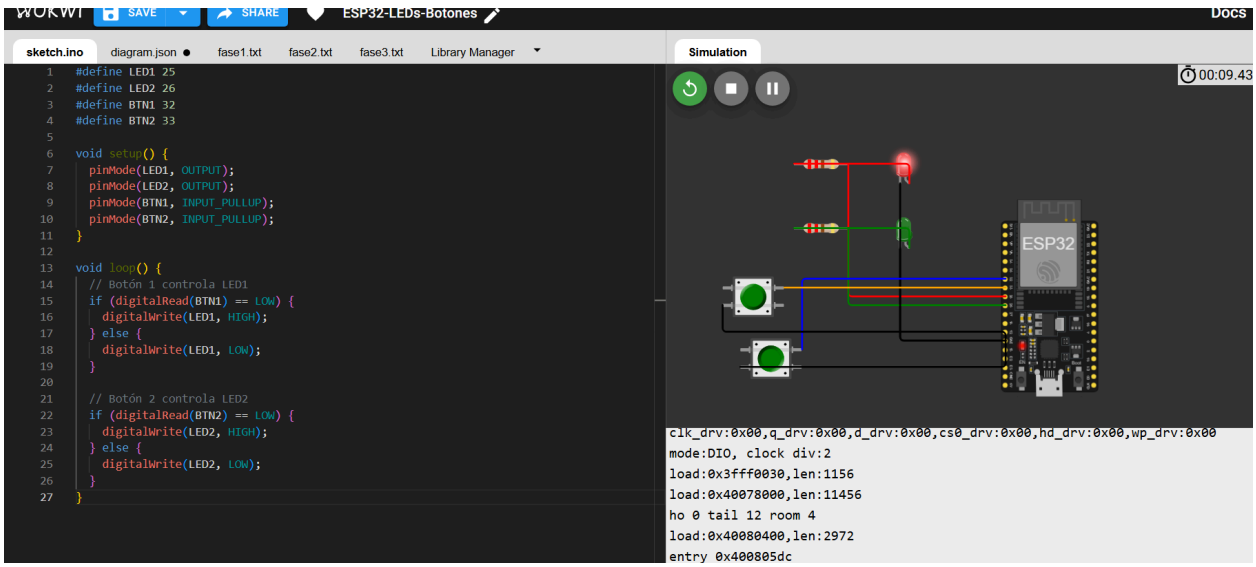

void setup() {
    pinMode(Config::PIN_LED_FIJO, OUTPUT);
    pinMode(Config::PIN_LED_PARPADEO, OUTPUT);


    digitalWrite(Config::PIN_LED_FIJO, HIGH); // LED 1 Fijo
    digitalWrite(Config::PIN_LED_PARPADEO, LOW); // LED 2 Estado
inicial
}
```

```
void loop() {
    actualizarLedParpadeante(); // Bucle libre sin atrasos por delay()
}
```

Fase 3





Codigo fase 3

```
/**
 * @file Fase3_Mapeo_Entradas_Salidas.cpp
 * @brief Práctica 1 - Fase 3: Lectura de entradas digitales e inversión de lógica Pull-Up.
 * @author Aguilar Nova Eric Oswaldo
 * @date 2026B
```

```
 */

#include <Arduino.h>

namespace Config {
    // Salidas
    constexpr uint8_t PIN_LED1 {25};
    constexpr uint8_t PIN_LED2 {26};

    // Entradas
    constexpr uint8_t PIN_BTN1 {32};
    constexpr uint8_t PIN_BTN2 {33};
}

/**
 * @brief Lee los pulsadores y actualiza las salidas digitales correspondientes.
 */
void procesarEntradas() {
    // Lectura con lógica invertida debido a INPUT_PULLUP (LOW = Presionado)
    const bool btn1Presionado = (digitalRead(Config::PIN_BTN1) == LOW);
    const bool btn2Presionado = (digitalRead(Config::PIN_BTN2) == LOW);

    // Mapeo directo de estados a las salidas
    digitalWrite(Config::PIN_LED1, btn1Presionado ? HIGH : LOW);
    digitalWrite(Config::PIN_LED2, btn2Presionado ? HIGH : LOW);
}

void setup() {
    // Configuración de salidas
    pinMode(Config::PIN_LED1, OUTPUT);
    pinMode(Config::PIN_LED2, OUTPUT);
```

```
    // Configuración de entradas utilizando resistencias Pull-Up
internas
    pinMode(Config::PIN_BTN1, INPUT_PULLUP);
    pinMode(Config::PIN_BTN2, INPUT_PULLUP);
}


void loop() {
    procesarEntradas();
}
```

Reporte: Práctica 1 - ESP32, LEDs y Botones - IoT - Woki(ESP32)

Fase 1
Se utiliza el espacio de nombres `Config` para definir el pin asignado (`PIN_LED_PRINCIPAL` en GPIO25). En la función `setup()`, se declara el terminal como salida digital mediante `pinMode()` y se establece en estado lógico alto constante mediante `digitalWrite(..., HIGH)`. Dado que la salida no requiere cambios dinámicos, la función `loop()` permanece sin instrucciones de ejecución.

En esta parte aprendí sobre el manejo básico de registros de dirección de pines (GPIO) como salidas digitales estáticas y comprensión del flujo de inicialización en sistemas embebidos.

Fase 2

En lugar de suspender la ejecución del procesador utilizando `delay()`, se implementa temporización asíncrona mediante la función `millis()`. El primer LED permanece encendido de forma continua desde `setup()`. En el bucle principal (`loop()`), se evalúa periódicamente la diferencia entre el tiempo actual y el último cambio de estado (`tiempoActual - tiempoAnterior`). Al alcanzar el umbral de 3000 ms, se invierte la variable de estado y se actualiza el valor del segundo LED en GPIO26.

Gracias a esta fase, me quedó un poco más clara la implementación de tareas no bloqueantes para multitarea en microcontroladores, evitando el congelamiento de la CPU y garantizando alta disponibilidad del sistema.

Fase 3

Los pines de entrada (GPIO32 y GPIO33) se configuran en modo `INPUT_PULLUP`, activando las resistencias internas conectadas a V_{CC}. Esta configuración invierte la lógica digital: el pin lee un estado alto (`HIGH`) en reposo y un estado bajo (`LOW`) al presionar el pulsador (conexión a tierra). La función modular `procesarEntradas()` consulta el estado de las entradas e invierte el valor lógico para activar los LEDs correspondientes (GPIO25 y GPIO26) solo mientras el botón permanezca oprimido.

Aprendí sobre la lectura e interpretación de entradas digitales con lógica Pull-Up, acondicionamiento de señales físicas mediante software y diseño modular fuertemente tipado.